Behavioural Fingerprinting of Honeypot and Flow Telemetry for Returning-Attacker Recognition and Adaptive Firewall Rule Generation

Saksham Mishra, Sahil Mohril, Ramyash Gupta
School of Computer Science Engineering

***Abstract*—Honeypots are increasingly used as the sensing layer of adaptive defence, but most systems either profile attackers without acting on the result or act on flow-level signatures without a behavioural model. We present an end-to-end prototype that closes this loop on recorded telemetry. Four input adapters map CICFlowMeter CSVs (CIC-IDS2017, CIC-DDoS2019), Cowrie and OpenCanary JSON logs and Zeek `conn.log` files onto one event schema. A label-free view of that schema feeds every detection component, and only the evaluator reads dataset labels. A deterministic replay engine streams slices of these sources at 1–100×. It can split one attack class in time and replay its second part from consistently remapped IP addresses to emulate a returning attacker. Sliding-window extraction computes 49 flow-aware features per source IP and per group of sparse sources; features that a source cannot observe are left undefined rather than set to zero. HDBSCAN discovers behaviour clusters without labels and turns them into fingerprints: a centroid, a MinHash signature over categorical traits, per-episode source sets and an explainable rule-based category. Live windows are matched by a weighted sum of anchored cosine similarity and per-family Jaccard containment. A new episode of a fingerprint whose sources are disjoint from all earlier episodes raises a *returning-behaviour* alert. Matches generate nftables, iptables and Suricata rules with a suggested–approved–active–expired lifecycle, an allowlist invariant and an in-app simulated firewall. We evaluate the prototype on format-faithful sample files (20–50 records per source, looped by the replay engine), not on the full datasets. Under that caveat, the returning attacker was recognised in 40 of 42 replays (mean similarity 0.955), each with an alert about 6.8 s after its return. A mixed eight-source replay reached an attack-versus-benign F1 of 0.926 at precision 1.0. The pipeline processed 9,187 events/s and the simulated firewall 108,849 events/s, and no allowlisted event was ever dropped. The rule-based categoriser labelled only 3 of 7 returning classes correctly, which we trace to rate thresholds invalidated by time compression. All numbers are feasibility evidence, and one script regenerates them on the real datasets.**

***Index Terms*—Honeypot, behavioural fingerprinting, attacker profiling, HDBSCAN, MinHash, Isolation Forest, returning attacker, firewall rule generation, CIC-IDS2017, Cowrie, Zeek.**

I. Introduction

A honeypot has no legitimate users, so every contact with it is suspect, which removes at the source the false-positive problem of in-line anomaly detection [1], [2]. Over two decades the field has moved from single emulated hosts to hybrid honeynets, cloned decoys and LLM-assisted analysis [3]–[5]. A recent review of 22 primary studies [6] draws four conclusions that frame this work. Deterministic rule layers carry most of the measured detection accuracy: in one ablation an LLM working alone fell from 94.23% to 55.77% [5]. Behavioural descriptors such as frequency, duration, diversity and timing outlast payload signatures [7]–[9]. Realism matters more than interaction depth [4], [10]. And evaluation is fragmented by small samples, labels derived from the system under test and missing detection rates [6].

The review's capability matrix also shows a division of labour. Deception-focused work makes decoys convincing but leaves analysis to people. Profiling work characterises behaviour but rarely acts on it. Response work acts quickly, but on flow- or signature-level inputs instead of behavioural clusters. Only proposals such as IBAF [7] and AHIH [3] describe the full loop from telemetry to enforcement, and the one evaluated end-to-end framework used public datasets [11].

This paper describes and evaluates a working prototype of that loop. It operates on recorded honeypot logs and labelled flow datasets instead of live traffic, which makes every run reproducible and keeps ground truth independent of the detector. Our contributions are:

1) A **label-isolated ingestion layer**: adapters for four real formats, a single dataset-label-to-behaviour table that marks behaviours with no labelled source, and a typed boundary that keeps labels out of detection (Section IV).
2) A **deterministic replay engine** with a *returning-attacker* mode that splits an attack class in time and replays its second part from consistently remapped addresses with timestamps shifted to the present (Section V).
3) **Flow-aware behavioural fingerprinting**: 49 features with explicit missing-value semantics, HDBSCAN clustering, MinHash trait signatures, an explainable rule-based categoriser and an anchored cosine-plus-containment similarity (Sections VI–VII).
4) **Episode-based returning-behaviour detection** tied to rule generation in three formats, with a staged lifecycle, an allowlist invariant and a simulated firewall that shows the effect of enforcement (Sections VIII–IX).
5) A **reproducible evaluation** with ablations, throughput measurements and an explicit list of what was not measured (Sections X–XII).

II. Related Work

Deception platforms. DYNAMIT clusters production hosts with K-Medoids over a hybrid Manhattan–Jaccard distance and clones real hosts as containerised decoys; its decoys

returned the same service sets as their templates under an aggressive Nmap scan, while a stock T-Pot exposed more than 100 conflicting services [4]. Zhang et al. identified 589 Kippo and Cowrie honeypots among 30,000 Shodan SSH hosts [10]. Savage et al. ran high-, low- and no-deception SSH hosts side by side and found that a few addresses dominate session volume [8]. Our prototype does not contribute to decoy realism; it consumes what such decoys log.

Behavioural profiling. IBAF formalises session frequency, authentication rate, normalised command entropy, active days and ATT&CK vectors, clusters actors with K-Means and validates clusters against IoCs [7]. Lim and Hobert reduce about 112,000 SSH sessions to 176 patterns and 18 Ward clusters, 7 of which survive IoC validation [9]. Balogh et al. score attack patterns out of 420 and separate human CTF players (122 points on average) from Internet bots (41) [12]. Beltrán-López et al. aggregate CICFlowMeter flows per attacker into 90 features and infer 24 ontology attributes with F1-macro between 0.63 and 0.999 [11]. Earlier work showed that statistical fingerprints of packet size and timing expose tunnels inside allowed protocols [13], and that support- and lift-based profiles identify individual users from sessions [14]. We adopt the flow-aggregation view of [11] and the log-transform-and-standardise practice of [7], [8], but cluster with HDBSCAN [15] because the densities of the behaviours we observe differ by orders of magnitude.

Learning and LLM analysis. The grounded honeypot–LLM pipeline of Oliveira et al. reached 94.23% accuracy on 52 observations, but its rules alone matched that figure [5]. The Double-DQN IPS of [16] reports 1.8 ms per-flow inference, and the dual-machine IPS of [17] stages blocks behind honeypot confirmation and canary rollback. Following the "deterministic first" finding, our decisions are rule- and similarity-based, with an Isolation Forest [18] only proposing candidate fingerprints.

Rule generation and enforcement. Saboori et al. mined Snort logs with Apriori and installed block rules above 70% support [19]. Pozo et al. diagnose inconsistent firewall rules on a platform-independent model in under 70 ms for 5,000 rules [20]. IBAF couples cluster-bound rules with SDN enforcement and Zero-Trust scores [7]. We generate rules per fingerprint episode, render them for three platforms and enforce them only in a simulated firewall.

Table I places this work on the five axes of [6], using the review's ratings for the other systems.

TABLE I
COVERAGE OF THE REVIEW'S FUNCTIONAL AXES [6]

System	A1	A2	A3	A4	A5	Validation
IDH [21]	•	○	○	○	•	Ransomware samples
DYNAMIT [4]	•	•	○	○	–	Synthetic nets
Honeypot–LLM [5]	○	–	•	•	○	52 observations
Real-time profiling [12]	•	–	•	○	–	CTF, Internet
IBAF [7]	•	○	•	•	•	Proposed protocol
Deception levels [8]	•	○	•	○	–	3 live hosts
Threat-actor ontology [11]	○	○	•	•	○	Public datasets
This work	○	–	•	•	•	Replayed samples

A1 deception platform; A2 attraction/anti-fingerprinting; A3 profiling; A4 learning; A5 response. • substantial, ○ partial, – not addressed.

III. System Overview

Fig. 1 shows the pipeline, aligned with the six-layer reference architecture of [6]. Files in a data folder are indexed by a catalog (labels, counts, time ranges). The replay engine streams selected slices through decoy services that fill in response details. A simulated firewall drops events that match active rules, and the capture layer persists events to SQLite and streams batched updates to a dashboard over WebSocket. Accepted events cross into the detection pipeline as label-free `ObservedEvent` records. Feature extraction, matching, clustering, alerting and rule generation run in one synchronous, deterministic orchestrator. A background thread drives it in real time, while tests and the experiment harness call it directly. The evaluator alone receives full events with labels.

IV. Data Model and Input Adapters

A. Event schema and label isolation

An event is $e = (t, src, dst, svc, proto, \mathbf{f}, \mathbf{h}, \mathbf{a}, \mathbf{c}, y, \kappa)$. It holds timestamps and endpoints; flow totals **f** (duration, forward and backward packets, bytes, SYN/ACK/FIN/RST/PSH counts); HTTP fields **h** (method, path, user agent, status); authentication fields **a** (username, password, attempt, outcome); and content **c** (command, payload snippet). The ground-truth label y and campaign identifier κ are kept verbatim from the source. Any field a source does not provide is `None`. The detection pipeline receives a separate type, `ObservedEvent`, which has no y or κ field, so a component cannot read a label even by mistake; a unit test checks this.

B. Adapters

CICFlowMeter CSV (CIC-IDS2017 [22], CIC-DDoS2019 [23]). Each row, a bidirectional flow, becomes one flow event. Headers are matched after stripping whitespace, against both the 2017 and the newer CICFlowMeter spellings. `Infinity` and `NaN` become missing. CIC-IDS2017 writes afternoon times on a 12-hour clock without AM/PM, so hours 1–7 are shifted by 12 h. Minute-resolution timestamps are spread evenly across the minute in file order. The variant without source IP and timestamp columns is rejected with an explanatory error. *Cowrie* [24]. Connect, login success and failure, command, download and session-close events are mapped, and source ports are carried from each session's connect event to its later events. *OpenCanary* [25]. Port-scan log types (5001–5005) become probe events whose flags are taken from the key set of the kernel log, and SSH, FTP, HTTP, MySQL and other login types become authentication attempts with an unresolved outcome. Boot and system messages are ignored. *Zeek conn.log* [26]. TSV with a `#fields` header or JSON lines are accepted. Flag counts are derived from the connection `history` string. IoT-23 `conn.log.labeled` files [27], whose last columns are joined by spaces, are split correctly, and their `detailed-label` becomes y.

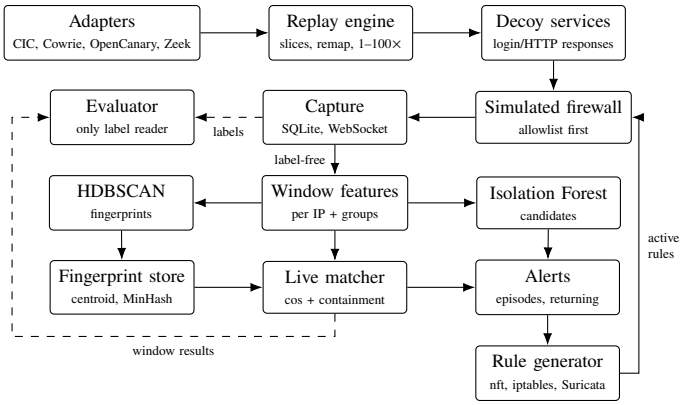


Fig. 1. Architecture. In the terms of [6], adapters and replay stand in for L1–L2, feature extraction for L3, HDBSCAN and the fingerprint store for L4, matching and alerts for L5, and rules with the simulated firewall for L6. Dashed edges carry ground truth and window results to the evaluator only.

TABLE II
DATASET-LABEL TO BEHAVIOUR MAPPING (EXCERPT)

Dataset labels	Behaviour
DoS Hulk, DoS GoldenEye, DDoS (2017), WebDDoS	HTTP flood
DoS slowloris, DoS Slowhttptest	Slowloris
PortScan; PartOfAHorizontal-PortScan (IoT-23)	Port scan (fast)
SSH-Patator / FTP-Patator	SSH / FTP brute force
Web Attack – Brute Force	Web login brute force
Web Attack – XSS, Sql Injection	Web vuln probing
Syn (2019)	SYN flood
DrDoS_*, UDP, UDP-lag, TFTP, LDAP, …	UDP flood / amplification
Bot; C&C*, Okiru (IoT-23)	Botnet / C2
Infiltration, Heartbleed	Infiltration
No data source: crawler, scraper, ICMP flood, stealthy scan, credential stuffing, post-login shell.	

C. Label mapping and coverage

One table maps normalised dataset labels to 19 behaviour keys (Table II). Behaviours that no supported labelled dataset provides are marked *no data source* instead of being simulated. These are search-engine crawler, aggressive scraper, ICMP flood and stealthy port scan, plus credential stuffing and post-login shell activity, which appear in Cowrie and OpenCanary logs but carry no label; a replay can attach a label to such a slice explicitly.

V. Deterministic Replay Engine

A *selection* names a source, optional label and time filters, an offset o on the replay timeline, a compression factor c (dataset seconds per replay second), a row cap, a sampling stride, an optional label override and an optional loop count. A dataset event at time t_d in a slice starting at t_0 is scheduled at replay time $r = o + (t_d - t_0)/c$ and stamped $t = b + r$, where b is the wall-clock time at which the replay starts, so replayed timestamps are shifted to the present. All selections are merged in a heap keyed by r. The engine advances in fixed steps of $\Delta = 0.25$ s regardless of wall-clock speed (Algorithm 1), so a plan always produces the same event sequence and the same results. A unit test checks that step sizes of 0.1, 1 and 5 s yield identical output. When the processing thread falls behind the requested speed, the replay clock slows down instead of skipping events.

Algorithm 1 Deterministic replay step

Input: heap Q of (r, k, e); clock r_c; base b; script Σ
$r_1 \leftarrow r_c + \Delta$; $E \leftarrow \emptyset$
while $Q \neq \emptyset$ **and** $\min(Q).r < r_1$ **do**
  $(r, k, e) \leftarrow \textsc{Pop}(Q)$; $e.t \leftarrow b + r$; $E \leftarrow E \cup \{e\}$
fire every unfired step $s \in \Sigma$ with $s.r < r_1$ ▷ narrate, mode, cluster
$r_c \leftarrow r_1$; **return** E

Algorithm 2 Returning-attacker plan

Input: records R of class ℓ sorted by time; split ϕ; target length T_A; gap g; baseline T_0; seed s
$n_A \leftarrow \lfloor \phi |R| \rfloor$; $A \leftarrow R_{1..n_A}$; $B \leftarrow R_{n_A+1..|R|}$
$m_A \leftarrow \lceil 60/|A| \rceil$, $m_B \leftarrow \lceil 60/|B| \rceil$ ▷ loop counts
$c \leftarrow \text{span}(A, m_A)/T_A$ ▷ shared compression
$o_A \leftarrow T_0$; $o_B \leftarrow o_A + \text{span}(A, m_A)/c + g$
add benign background spanning the plan, looped with fresh IPs if short
add (A, o_A, c, m_A) with campaign κ_A
add $(B, o_B, c, m_B, \text{remap} = \rho_s)$ with campaign κ_B, parent κ_A
script: force clustering at the end of A

Returning-attacker mode (Algorithm 2) takes the records of one attack class in time order and splits them at fraction ϕ. Part A replays with its original addresses after a benign baseline. Part B replays after a gap longer than the episode gap, with every source address a replaced by $\rho_s(a)$: a deterministic, injective-in-practice remapping derived from a keyed BLAKE2b hash with seed s, so all records from one original source keep one new address. Both parts share one compression factor, so their rates stay comparable. Slices too short to fill several windows can be looped back-to-back; benign background loops receive fresh addresses on each copy, so that a handful of hosts does not turn into periodic beacons. Every plan reports when this happened.

TABLE III
NUMERIC FEATURE FAMILIES (49 FEATURES)

Family	Features
Volume	events/s, packets/s, bytes/s, connections/s (log)
Protocol	TCP/UDP/ICMP ratio, SYN ratio, SYN:ACK, RST ratio, half-open ratio
Flow shape	duration, packets/flow, bytes/flow, reply ratio
Packets	size mean, size std, size entropy
Timing	IAT mean, IAT std, CV, burstiness
Breadth	unique dst ports, dst IPs, paths, users, passwords; src-port diversity
Auth	auth ratio, fail rate, fail ratio, success-after-fail, user diversity
HTTP	HTTP ratio, error ratio, UA diversity, path entropy, sensitive-path ratio
Content	command count, command ratio
Service	web, SSH/Telnet, FTP, DNS ratios
Sources	long-horizon ports, events, dst IPs; number of sources; rate per source

VI. Flow-Aware Behavioural Features

Every hop $H = 5$ s, events in the trailing window $W = 10$ s are grouped into *entities*. A source IP becomes an entity when it has at least $n_{\min} = 3$ events in the window or at least $4n_{\min}$ events in a 300 s history. Otherwise its events join a *campaign-cluster group* keyed by traffic shape: protocol, service, well-known port and a request class (SYN-only, RST, established, authentication, HTTP page or sensitive path, command). Groups with at least three events form an entity. This is how spoofed SYN floods and distributed credential attacks, which produce one or two events per source, become visible. For each entity we compute the 49 features of Table III. Count and rate features are transformed by $\tilde{x} = \log(1 + x)$ [7], [8]. Two examples:

$$\text{half_open} = |\mathcal{T}_{\text{ho}}| \,/\, |\mathcal{T}|, \tag{1}$$

$$\text{reply} = \log\big(1 + \text{mean}_{e\in\mathcal{F}}\, b_e/\max(1, f_e)\big), \tag{2}$$

where $\mathcal{T}$ are the TCP events with flag information, $\mathcal{T}_{\text{ho}} \subseteq \mathcal{T}$ those with SYN but no ACK and no reply packets, and f_e, b_e are the forward and backward packets of the flows $\mathcal{F}$ that report both. Inter-arrival statistics use the window, or the long history when the window has fewer than three events. We use $\text{CV} = \sigma/\mu$ and burstiness $B = (\sigma - \mu)/(\sigma + \mu)$.

Missing values. A feature that a source cannot observe is NaN, not zero. Flag ratios are undefined for Cowrie logins, and HTTP ratios are undefined for CIC flows. After scaling, NaN becomes 0, the baseline, so it contributes nothing to similarity, and the categoriser skips it. Treating "not observed" as "zero" would make, for example, every Cowrie window look like a perfect non-SYN-flood.

Categorical traits. Each entity also carries token sets in ten families: service and event type, top ports and a port-spread bucket, flag shapes, $\log_2$ size buckets, paths, user agents, usernames, passwords, command verbs and command bigrams per session, and payload tokens.

Algorithm 3 Clustering pass and fingerprint placement

Input: buffer $\mathcal{B}$; store $\mathcal{S}$; threshold θ

re-categorise fingerprints whose centroid drifted
$U \leftarrow \{w \in \mathcal{B} : w.fp = \varnothing\}$; refit spread on $\mathcal{B}$
for $w \in U$ **do** ▷ absorb leftovers
  $(F, s) \leftarrow \arg\max_{F\in\mathcal{S}_{\text{active}}} \text{sim}(w, F)$; **if** $s \geq \theta$ **then** UPDATE$(F, \{w\})$
$L \leftarrow \text{HDBSCAN}(z(U'), m_{\text{cs}}{=}5, m_{\text{s}}{=}3)$ on the remaining U'
for each cluster C with label ≥ 0 **do**
  $(F, s) \leftarrow \arg\max_{F\in\mathcal{S}} \text{sim}(\bar{C}, F)$ ▷ active, candidate, dismissed
  if $s \geq \theta - 0.05$ **then**
    if F dismissed **then** mark C suppressed
    else UPDATE(F, C); promote F if candidate and $|C| \geq m_{\text{cs}}$
  else $F \leftarrow$ CREATE(C): centroid, spread, tokens, MinHash, category, episode

VII. Fingerprinting

A. Scaling

A StandardScaler fitted on the recent window buffer supplies the spread, floored at 0.2 so that near-constant features cannot dominate. The centre is the mean of a benign baseline period (the first 30 s of a replay), so normal traffic sits near the origin:

$$z_j = \text{clip}\Big(\frac{x_j - \mu_j^{\text{base}}}{\max(\sigma_j^{\text{buf}}, 0.2)}, -8, 8\Big), \qquad z_j \leftarrow 0 \text{ if } x_j = \text{NaN}. \tag{3}$$

B. Clustering: HDBSCAN rather than DBSCAN

Behaviour densities differ by orders of magnitude: a flood produces hundreds of near-identical windows, a scan a handful. DBSCAN's single ε either merges sparse groups or splits dense ones. HDBSCAN [15] keeps the most stable clusters across densities, needs no k, and labels leftovers as noise, which remain unassigned and may become anomaly candidates. Every 15 s, the clustering pass in Algorithm 3 runs on unassigned windows only. Before clustering, it rematches leftover windows against existing fingerprints, so that stragglers from an already-known behaviour do not seed a mixed cluster with a newer one. We found this failure mode during development.

C. Fingerprint model

A fingerprint stores a raw centroid and spread, token counts per family (at most 400), a 128-permutation MinHash signature, importance as z-scores against the baseline, source, ASN and country counters, a list of episodes, a version history and a hit timeline. MinHash uses $h_i(x) = \text{SplitMix64}(\text{hash}(x) \oplus s_i)$ with 128 random seeds [28], [29]. An earlier variant of the form $(ax+b) \bmod p$, with 31-bit coefficients on 32-bit hashes, barely wraps modulo p, so it is close to a monotone map. It estimated a true Jaccard of 1/3 as 0.16, and the SplitMix64 version passes the same test. Live matches nudge the centroid with an exponential moving average ($\alpha = 0.02$) and add tokens, so fingerprints keep learning.

D. Explainable categoriser

Each behaviour has one or more condition lists over centroid features (e.g. SYN flood: half-open ratio ≥ 0.7; packets/flow ≤ 3; $\geq$20 flows/s; random source ports). The first condition, and any marked as required, must be observable and satisfied. Let E be the observable conditions of a list with n conditions and $M \subseteq E$ those satisfied. The confidence is

$$\gamma = \frac{|M|}{|E|}\Big(0.85 + 0.15\frac{|E|}{n}\Big), \quad |E| \geq n/2, \tag{4}$$

rounded to four decimals. Without rounding, a 3-of-4 score falls just under the 0.75 cut-off in floating point, which we observed. The best list wins if $\gamma \geq 0.75$. Otherwise the fingerprint is *benign* when the mean $|z|$ of its centroid against the baseline is below 1, and *unknown anomaly* when it is not. The matched and unmatched condition texts are kept as the explanation.

E. Similarity

For a live window w and a fingerprint F with scaled vectors $\mathbf{z}_w, \mathbf{z}_F$:

$$\cos^*(w,F) = \max\Big(0, \frac{[\mathbf{z}_w;\eta]\cdot[\mathbf{z}_F;\eta]}{\|[\mathbf{z}_w;\eta]\|\,\|[\mathbf{z}_F;\eta]\|}\Big), \ \eta = 2, \tag{5}$$

$$J^{\subset}(w,F) = \frac{1}{|\Phi_w|}\sum_{\phi\in\Phi_w}\frac{|T_w^\phi \cap T_F^\phi|}{|T_w^\phi|}, \tag{6}$$

$$S(w,F) = \frac{w_n\cos^* + w_c J^{\subset}}{w_n + w_c}, \quad w_n = 0.6,\ w_c = 0.4. \tag{7}$$

The constant *anchor* dimension η in (5) makes low-energy, baseline-like vectors point along the anchor instead of in arbitrary directions, while strong behaviours are compared by the direction of their deviation. A replay at a different rate along the same direction therefore keeps a high score. A 10 s window holds a small sample of a fingerprint's traits, so plain Jaccard would penalise it for being small. We therefore use containment (6), averaged over the token families Φ_w present in the window, and keep the MinHash Jaccard estimate for fingerprint-to-fingerprint comparison. When a window has no tokens, $S = \cos^*$. Every match records its explanation: per-feature contributions $z_{w,j}z_{F,j}/(\|\cdot\|\|\cdot\|)$, the most divergent features, containment per family, and known versus new tokens.

VIII. Detection, Episodes and Alerts

Algorithm 4 processes each hop. A window whose best active fingerprint scores $S \geq \theta = 0.85$ is a match. Otherwise an Isolation Forest [18], trained on the baseline windows (150 trees, contamination 0.03), flags anomalies. An anomalous window joins a similar candidate fingerprint ($S \geq \theta - 0.1$) or creates one, at most three per hop; candidates are promoted when a dense cluster confirms them, or by the user.

Episodes and returning behaviour. An episode of F is a maximal run of matches separated by less than a gap $g_e = 25$ s. When a match opens a new episode ϵ_k with source set I_k,

$$\text{returning}(\epsilon_k) \iff k > 1 \ \wedge \ I_k \cap \bigcup_{j<k} I_j = \varnothing. \tag{8}$$

Algorithm 4 Live matching, episodes and returning detection

Input: windows $\mathcal{W}$ at time t; active fingerprints $\mathcal{S}$; IF model
for $w \in \mathcal{W}$ **do**
  $(F, s) \leftarrow \arg\max_{F\in\mathcal{S}} S(w,F)$
  if $s \geq \theta$ **then**
    $\epsilon \leftarrow$ current episode of F
    if ϵ exists **and** $t - \epsilon.end \leq g_e$ **then** $\epsilon.I \leftarrow \epsilon.I \cup w.I$
    else open ϵ' with $I = w.I$; $\epsilon'.ret \leftarrow$ condition (8)
    raise RETURNING if $\epsilon.ret$ else MATCH; RULEFOR(F, ϵ)
  else if IF$(z_w) < 0$ **then**
    join or create candidate; raise ANOMALY
  append w to the clustering buffer

TABLE IV
RULE STRATEGY PER BEHAVIOUR CATEGORY

Category	Rule kind
SYN / UDP / ICMP flood	SYN rate limit / UDP port limit / ICMP limit
HTTP flood, credential stuffing	IP-set block with timeout
Slowloris	connection limit + source block
Scans, SSH/FTP/web brute force, C2	IP block (IP set above 20 sources)
Web probing, post-login shell, malware	Suricata content + IP block
Unknown anomaly	IP block, suggest-only

The behaviour is then known but its sources are not, which is the signature of an attacker who rotated infrastructure [7], [8].

Alerts are keyed by (type, fingerprint, episode) and deduplicated within 120 s. The severity rank of an alert hit n times is

$$\text{sev}(n) = \min\big(\text{sev}_0 + [n \geq 10] + [n \geq 50],\ \text{CRITICAL}\big). \tag{9}$$

Returning alerts start at least at HIGH. Categories that resolve to benign never alert.

IX. Rule Generation and Simulated Enforcement

Each (fingerprint, episode) gets at most one rule, whose kind depends on the category (Table IV). The rule records its fingerprint, reason, creation time, expiry and history. It is rendered as nftables (sets with timeouts, meters), iptables (ipset, `hashlimit`, `connlimit`) and Suricata (`threshold`, `http.uri` and `http.user_agent` content). New sources of the same episode extend the rule instead of creating another. The lifecycle is *suggested→approved→active→expired/rejected*. In suggest mode an analyst approves rules; in auto mode they activate immediately, except unknown-anomaly rules, which are never auto-applied (cf. staged enforcement in [17]).

Allowlist invariant. Allowlisted CIDRs are removed from every rule target, including later user edits. The simulated firewall checks the allowlist before any rule, including rate limits, so an allowlisted source is never dropped. Rate rules use token buckets: with rate λ and burst 2λ, an event at time t passes if $\min(2\lambda, b + \lambda(t - t')) \geq 1$, where b is the bucket level at the previous event t'. Dropped events are counted per rule and kept out of detection, which lets the dashboard show traffic before and after enforcement. No rule is ever executed on a host.

TABLE V
E1: Field coverage per adapter (share of records with the field)

File	Rec.	Cls.	Pkts	Dur.	Flags	Cred.	Cmd/Path
CIC-IDS2017 CSV	50	6	1.00	0.98	0.80	–	–
CIC-DDoS2019 CSV	40	3	1.00	1.00	0.63	–	–
Cowrie JSON	28	–	–	0.07	0.07	0.54	0.25
OpenCanary JSON	27	–	0.37	–	0.37	0.41	0.22
Zeek TSV (IoT-23)	30	3	1.00	0.40	0.80	–	–
Zeek JSON	20	–	1.00	0.75	1.00	–	–

Cls.: labelled classes. Cred.: username present. Cmd/Path: command (Cowrie) or HTTP path (OpenCanary).

TABLE VI
E2: Returning-attacker recognition (6 runs per class)

Class (source)	Rec.	$\bar{S}$ (min)	Delay	Drop	Category
SSH-Patator (2017)	6/6	0.944 (0.885)	5.0 s	0.85	SSH brute force
FTP-Patator (2017)	6/6	0.971 (0.961)	5.0 s	0.92	FTP brute force
PortScan (2017)	4/6	0.974 (0.972)	5.0 s	0.75	C2 (wrong)
Syn (2019)	6/6	0.968 (0.956)	5.0 s	0.91	C2 (wrong)
DrDoS_DNS (2019)	6/6	0.968 (0.957)	16.7 s	0.73	C2 (wrong)
Horiz. scan (IoT-23)	6/6	0.942 (0.928)	5.0 s	0.91	SSH b.f. (wrong)
C&C-HeartBeat (IoT-23)	6/6	0.924 (0.920)	5.0 s	0.92	C2
All	40/42	0.955 (0.885)	6.8 s	0.88	3/7 classes

Rec.: part B matched the part-A fingerprint; every recognised run also raised a returning alert. Delay: part-B start to alert. Drop: share of part-B events dropped.

X. Evaluation Methodology

Data. The full datasets were not available when we ran these experiments, so all results use the bundled sample files: two CICFlowMeter CSVs with real CIC-IDS2017 and CIC-DDoS2019 headers (50 and 40 rows), a Cowrie log (28 events), an OpenCanary log (27), an IoT-23-style labelled Zeek TSV (30) and a Zeek JSON log (20). They are *synthesised* in each tool's real format, with labels drawn from the datasets' vocabularies, and the replay engine loops them. The review warns about such ground truth [6]: these labels are independent of the detector, but the data is small, synthetic and repetitive. Looping produces artefacts that real data would not have, such as repeated source ports and perfectly periodic timing.

Label provenance. Window ground truth is the majority mapped label of the events a window contains. Cowrie and OpenCanary windows are unlabelled and excluded unless overridden.

Metrics. We report window-level precision, recall and F1 per behaviour, and attack-versus-benign, both *online* (at detection time) and *after clustering*, which credits windows that HDBSCAN later assigned to a fingerprint. We also report the mean time to detect (first event of a campaign to its first flagged window) and the time to rule; returning-attacker recognition, meaning that a part-B window matched a fingerprint that part-A windows built or matched; and alert raised, similarity, alert delay and the share of part-B events dropped. Finally we report throughput and per-stage latency.

Setup. An Intel Core i5-1235U laptop with 15.7 GB RAM, Windows 11, Python 3.11.14, scikit-learn 1.5.2 and NumPy 2.1.3. Default settings throughout (W = 10 s, H = 5 s, θ = 0.85, w_n:w_c = 0.6:0.4, HDBSCAN m_{cs} = 5, m_s = 3). Every run is deterministic, so variation comes only from the split fraction and the remap seed. `python -m experiments.paper_experiments` regenerates all tables.

XI. Results

A. E1: Adapter coverage

Table V shows which fields each format actually provides. Flow sources fill the flow totals almost completely. Honeypot logs fill credentials, commands and HTTP fields but no flow totals, which is why features are defined per field. Parsing costs a fixed 0.1–0.18 s per CSV file, dominated by pandas set-up, and 6–10 ms per JSON or TSV file of this size.

B. E2: Returning-attacker recognition

We ran the returning-attacker plan for every labelled attack class in the samples with split fractions $\phi \in \{0.3, 0.5, 0.7\}$ and two remap seeds, 42 runs in all (Table VI). Part B was recognised, meaning that it matched the fingerprint built from part A although every source address had changed, in 40 runs, with a returning-behaviour alert in each of them. The mean similarity of the first returning match was 0.955 (range 0.885–0.990), the alert came on average 6.8 s after part B started (one or two hops), and auto-applied rules dropped 87.7% of part-B events on average. Both failures were PortScan with $\phi = 0.3$, where part A contained only 2 of the 8 sample records and never formed a cluster. Part A was first detected 33.8 s and first ruled 30.2 s after it started, on average. Time to rule is shorter than time to detect because a rule is created when clustering forms the fingerprint, while the first *online* detection waits for the next hop.

The categoriser, however, assigned the expected category in only 3 of 7 classes (SSH, FTP and C&C). The returning plan compresses part A to a fixed 60 s regardless of its original rate, so absolute-rate conditions see rates that the original traffic never had: a looped SYN slice at about one flow per second looks like low-rate periodic beaconing. In the mixed run below, where SYN traffic keeps its sample rate, it was categorised correctly. Recognition does not depend on the category because it uses the similarity (7), but the label shown to an analyst does.

C. E3: Mixed replay

Eight slices from all six files ran overlapping in one plan: 7,528 events and 836 labelled windows. They yielded 18 fingerprints, 9 of them non-benign, covering SYN flood, UDP flood, SSH brute force, fast port scan, C2, web probing (from the unlabelled OpenCanary log) and unknown anomaly. Attack-versus-benign detection had TP = 69, FP = 0, FN = 11, TN = 756, giving precision 1.000, recall 0.863 and F1 0.926. Per-behaviour results are in Table VII. Online recall is low because windows observed before the first clustering pass are unmatched by construction. After clustering, the macro F1 was 0.995. The mean time to detect was 10.0 s and to rule 30.0 s. These per-class precisions of 1.0 partly reflect that the

TABLE VII
E3: PER-BEHAVIOUR WINDOW SCORES ON THE MIXED REPLAY

		Online		After clustering	
Behaviour	Supp.	Rec.	F1	Rec.	F1
Benign	756	0.806	0.892	0.999	0.999
SSH brute force	39	0.615	0.762	0.949	0.974
Botnet / C2	13	0.615	0.762	1.000	1.000
Port scan (fast)	11	0.455	0.625	1.000	1.000
UDP flood	10	0.300	0.462	1.000	1.000
SYN flood	7	0.143	0.250	1.000	1.000
Macro (P = 1.000)	836	0.489	0.626	0.991	0.995

TABLE VIII
E4: ABLATIONS (MIXED REPLAY; 7 RETURNING CASES AT $\phi = 0.5$)

Variant	$F1_{bin}$	$Rec._{bin}$	$mF1_{on}$	$mF1_{fin}$	Ret.
Full (θ=0.85, 0.6/0.4, η=2)	0.926	0.863	0.626	0.995	7/7
Numeric only (w_c=0)	0.912	0.838	0.607	0.991	7/7
Categorical only (w_n=0)	0.940	0.887	0.632	1.000	5/7
No anchor (η=0)	0.926	0.863	0.626	0.995	7/7
$\theta = 0.75$	0.926	0.863	0.626	0.996	7/7
$\theta = 0.90$	0.912	0.838	0.494	0.835	7/7
$\theta = 0.95$	0.873	0.775	0.555	0.980	5/7
DBSCAN $\varepsilon = 0.5$	0.788	0.650	0.438	0.981	7/7
DBSCAN $\varepsilon = 1.0$	0.881	0.787	0.400	0.989	7/7
DBSCAN $\varepsilon = 2.0$	0.881	0.787	0.400	0.989	7/7

Precision was 1.000 in every variant. Ret.: returning attackers recognised.

sample classes come from different sources and hardly overlap in feature space.

D. E4: Ablations

Table VIII varies one design choice at a time on the mixed replay and on the seven returning cases ($\phi = 0.5$).

- *Categorical containment alone* slightly raised mixed-run F1 (0.940) but recognised only 5 of 7 returning attackers. Remapping changes no token, so containment is high for every window of an already-seen service, and the weaker numeric term no longer separated the cases.
- *Numeric cosine alone* recognised all 7 at a slightly lower F1 (0.912). The combination keeps both properties.
- *The anchor dimension* made no measurable difference on this data (mean score 0.961 vs. 0.964); its intended effect concerns low-energy benign windows, which the small benign sample does not exercise.
- *Raising* θ to 0.90 cut the after-clustering macro F1 from 0.995 to 0.835; at 0.95 two returning attackers were missed. Lowering it to 0.75–0.80 changed nothing measurable.
- *DBSCAN* in place of HDBSCAN lowered attack recall from 0.863 to 0.650–0.787 and online macro F1 from 0.626 to 0.400–0.438 for $\varepsilon \in \{0.5, 1, 2\}$, consistent with the density argument of Section VII.

E. E5: Scripted demonstration

The deterministic demo replays benign background, SSH-Patator part A, a switch to auto-apply, and part B from new addresses (Fig. 2). The baseline was calibrated on 144 windows at $t = 30$ s. The SSH fingerprint formed at $t = 60$ s, when its rule was *suggested*. Part B started at $t = 110.2$ s, and 4.8 s later the returning alert fired with similarity 0.89, the rule was auto-applied and traffic from the new source began to be dropped (58 of 2,123 events). Four benign windows raised low-severity anomaly candidates, which gives attack-versus-benign precision 0.636 on this small run.

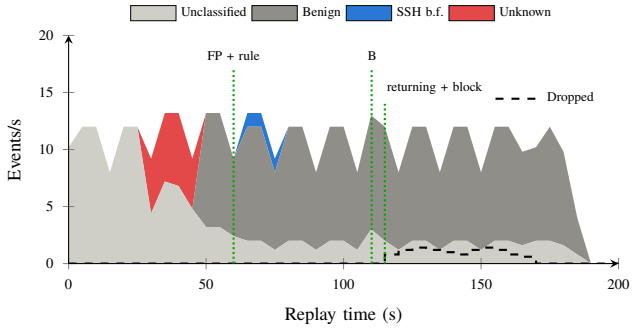


Fig. 2. Demonstration timeline (5 s bins): accepted events per second by the detected category of their source (stacked) and events dropped by the simulated firewall (dashed). Markers: fingerprint and suggested rule (60 s), part B returns from new IPs (110.2 s), returning alert and auto-block (115 s).

TABLE IX
E6: MEASURED THROUGHPUT AND LATENCY

Measurement	Result
End-to-end headless pipeline	9,187 events/s (197× real time)
Feature extraction per hop (mean / P95)	6.9 / 18.0 ms
Live matching per hop (mean / P95)	11.6 / 18.0 ms
Clustering pass (mean / P95, 85 runs)	2.7 / 5.0 ms
HDBSCAN, 250 / 500 / 1,000 / 2,000 windows	0.011 / 0.031 / 0.091 / 0.383 s
Simulated firewall, 16 active rules	108,849 events/s
Allowlisted events dropped	0 of 50,000
Rule rendering, three formats	≤0.03 ms per rule
Live API latency under 100× replay	17–72 ms

F. E6: Throughput and latency

Table IX lists the measured costs. The headless pipeline processed 60,980 events of a 1,306 s replay in 6.64 s, i.e. 9,187 events/s or 197× real time, with feature extraction at 6.9 ms (P95 18.0 ms) and matching at 11.6 ms (P95 18.0 ms) per 5 s hop. HDBSCAN on the window buffer grew from 0.011 s at 250 windows to 0.383 s at 2,000, roughly quadratic in this range, which caps the default buffer at 2,500 windows. With 16 active rules of six kinds, the simulated firewall checked 108,849 events/s; of 200,000 events, 50,000 came from allowlisted sources and none was dropped. Rendering a rule in three formats took at most 0.03 ms. In the live application with SQLite persistence and WebSocket streaming at a requested 100×, the backend processed about 18,600 events in the first second and answered REST requests in 17–72 ms. For orientation only, since hardware and workloads differ: the Double-DQN IPS of [16] reports 1.8 ms per flow, and the grounded LLM pipeline of [5] reports 24.1 s mean inference per assessment.

XII. Discussion

Behaviour as the unit of response. The returning-attacker result supports the review's claim that behavioural descriptors survive infrastructure rotation [6], [7]: changing every source address left the fingerprint match intact in 40 of 42 runs, and the episode rule turned that match into an actionable alert within one or two hops. Binding rules to fingerprint episodes gives a unit that, as IBAF argues [7], is more stable than an address yet still fine-grained enough to choose a rule kind.

Deterministic first. All decisions in the prototype are rule- or similarity-based. The learned components (HDBSCAN and the Isolation Forest) only propose structure, and every decision carries an explanation. This follows the ablation evidence of [5]. Our own ablations point the same way: the categorical term helps separate behaviours but cannot recognise a returning attacker on its own.

Where the categoriser fails. Absolute rate thresholds are fragile under time compression (E2). A deployment on live telemetry does not compress time, but replay-based evaluation does, and so would a deployment whose window size differs from the one the thresholds were written for. Rate conditions should be expressed relative to the baseline, for example as z-scores, or learned from labelled data, with the hand-written rules kept as explanations, as [11] does with SHAP.

Maturity. In the five-level maturity model of [6], the prototype implements level-3 analysis (correlated detection, explanations, rule-based response) and parts of level 4: staged rules, an allowlist invariant, fingerprints that update with evidence and drift re-categorisation. It has no adaptive deception, no verified rule-set consistency [20] and no live traffic.

What was not measured. Following the review's recommendation to report unmeasured quantities next to headline numbers [6], we did not measure:

- any result on the full CIC-IDS2017, CIC-DDoS2019 or IoT-23 data, or on real Cowrie and OpenCanary logs;
- robustness to an adaptive adversary that shapes its traffic into benign clusters or poisons the buffer;
- end-to-end latency on live traffic, or the effect of enforcement on a real network;
- the consistency of generated rule sets with existing firewall policy [20];
- the variance of results beyond the split and seed sweep, since every run is deterministic.

Threats to validity. The samples are synthesised, small and looped, so absolute scores, especially the per-class precision of 1.0, overstate what real traffic will yield. Real CIC data reuses one attacker address across classes, so per-IP windows can mix behaviours. The category ground truth relies on our label table. The thresholds were tuned during development on the same samples.

XIII. Conclusion and Future Work

We presented a reproducible, label-isolated prototype that turns honeypot logs and labelled flows into behavioural fingerprints, recognises the same behaviour when it returns from new addresses, and generates staged, allowlist-safe firewall rules in three formats. On sample data it recognised returning attackers in 40 of 42 replays within one or two hops, reached an attack-versus-benign F1 of 0.926 at precision 1.0, and ran at about 9,000 events/s. The next steps follow directly from the gaps: running the experiment harness on the full datasets and on live Cowrie, OpenCanary and Zeek feeds; replacing absolute-rate rules with baseline-relative or learned thresholds; adding consistency checks [20] and canary rollback [17] before enforcement; and evaluating against adaptive adversaries, in line with the agenda of [6].

References

[1] L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2002.

[2] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 3, pp. 186–205, 2000.

[3] A. K. Aseres, S. Z. Melese, A. M. Beyene, and L. K. Tegegne, "High-interaction honeypots in cyber deception: Evolution, design, deployment, and future trends," *IEEE Access*, vol. 14, pp. 99861–99896, 2026, doi: 10.1109/ACCESS.2026.3706613.

[4] Y. Maraden, Z. Ananda, I. G. D. Nugraha, and R. F. Sari, "DYNAMIT: K-Medoids-based machine learning for scalable honeynet deception and intelligent threat profiling," *Electronics*, vol. 15, no. 11, Art. no. 2490, 2026, doi: 10.3390/electronics15112490.

[5] R. Oliveira, T. Gomes, D. Pinho, and F. Almeida, "An integrated honeypot and LLM-based framework for near real-time detection and behavioral analysis of malicious activities," *J. Cybersecur. Priv.*, vol. 6, no. 5, Art. no. 147, 2026, doi: 10.3390/jcp6050147.

[6] S. Mishra, S. Mohril, and R. Gupta, "Honeypots and behavioural intelligence for adaptive cyber defence: A comprehensive review," manuscript, 2026.

[7] S. Mohril, "An intelligent behavioural analysis framework for adaptive network defence using behavioural fingerprinting," manuscript, 2026.

[8] E. Savage, N. D. Bastian, and R. Zhou, "Attack behavior observations and profiling through varying cyber deception mechanisms," in *Proc. IEEE MILCOM*, 2025, pp. 758–763, doi: 10.1109/MILCOM64451.2025.11310496.

[9] T. K. Lim and B. Hobert, "Enhancing honeypot-based cyber attribution using hierarchical clustering," *Digital Threats: Research and Practice*, ACM, 2026.

[10] Y.-J. Zhang, W.-J. Liu, K.-N. Guo, and Y.-M. Kang, "Identification of SSH honeypots using machine learning techniques based on multi-fingerprinting," in *Proc. IEEE 6th ITNEC*, 2023, pp. 1376–1381, doi: 10.1109/ITNEC56291.2023.10082467.

[11] P. Beltrán-López, M. Gil Pérez, and P. Nespoli, "Enhancing strategic decision-making via semantic inference: An adaptive framework for threat actor profiling," *Inf. Fusion*, vol. 134, Art. no. 104359, 2026, doi: 10.1016/j.inffus.2026.104359.

[12] Á. Balogh, E. Kail, M. Érsők, and A. Bánáti, "Possibilities for optimization of real time attacker profiling with honeypots," in *Proc. IEEE 19th SACI*, 2025, pp. 111–116, doi: 10.1109/SACI66288.2025.11030113.

[13] M. Dusi, M. Crotti, F. Gringoli, and L. Salgarelli, "Tunnel Hunter: Detecting application-layer tunnels with statistical fingerprinting," *Comput. Netw.*, vol. 53, pp. 81–97, 2009, doi: 10.1016/j.comnet.2008.09.010.

[14] Y. (C.) Yang, "Web user behavioral profiling for user identification," *Decis. Support Syst.*, vol. 49, pp. 261–271, 2010, doi: 10.1016/j.dss.2010.03.001.

[15] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. PAKDD*, LNCS vol. 7819, 2013, pp. 160–172.

[16] S. V. Mallikarjunaradhya, R. Pavithra, N. Sanjay, J. C. Sriram, and S. V. S. Yadav, "Adaptive real-time intrusion prevention framework using Double DQN and prioritized experience replay," in *Proc. ICAUC*, IEEE, 2026.

[17] S. S. Panwar *et al.*, "A dual-machine intrusion prevention system architecture with base-rate-fallacy mitigation," IEEE.

[18] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE ICDM*, 2008, pp. 413–422.

[19] E. Saboori, S. Parsazad, and Y. Sanatkhani, "Automatic firewall rules generator for anomaly detection systems with Apriori algorithm," in *Proc. 3rd ICACTE*, 2010, vol. 6, pp. V6-57–V6-60.
[20] S. Pozo, R. Ceballos, and R. M. Gasca, "Model-based development of firewall rule sets: Diagnosing model inconsistencies," *Inf. Softw. Technol.*, vol. 51, pp. 894–915, 2009, doi: 10.1016/j.infsof.2008.05.001.
[21] S. Sibi Chakkaravarthy, D. Sangeetha, M. Vimal Cruz, V. Vaidehi, and B. Raman, "Design of intrusion detection honeypot using social leopard algorithm to detect IoT ransomware attacks," *IEEE Access*, vol. 8, pp. 169944–169956, 2020, doi: 10.1109/ACCESS.2020.3023764.
[22] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th ICISSP*, 2018, pp. 108–116.
[23] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. IEEE ICCST*, 2019, pp. 1–8.
[24] Cowrie SSH/Telnet honeypot. [Online]. Available: https://docs.cowrie.org/
[25] Thinkst Applied Research, "OpenCanary." [Online]. Available: https://github.com/thinkst/opencanary
[26] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Comput. Netw.*, vol. 31, no. 23–24, pp. 2435–2463, 1999.
[27] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, 2020, doi: 10.5281/zenodo.4743746.
[28] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997, pp. 21–29.
[29] G. L. Steele Jr., D. Lea, and C. H. Flood, "Fast splittable pseudorandom number generators," in *Proc. ACM OOPSLA*, 2014, pp. 453–472.